Name: zain pawle — **Roll No:** 25dco08
Batch: 3 — **Class:** TECO1

Experiment Report: Sentiment Analysis Using Pretrained BERT Model

1 Aim

To implement a Sentiment Analysis application using a pretrained BERT model (`textattack/bert-bas`) with PyTorch, a FastAPI web backend, and an HTML/CSS/JS frontend interface.

2 Theory

Bidirectional Encoder Representations from Transformers (BERT) is a transformer-based machine learning framework developed by Google for Natural Language Processing (NLP). Standard sequential models (like LSTMs) read text left-to-right or right-to-left sequentially. In contrast, BERT reads the entire sequence of words at once using bidirectional attention mechanisms.

In this project, the `textattack/bert-base-uncased-SST-2` model—a BERT base model fine-tuned specifically on the Stanford Sentiment Treebank (SST-2) dataset—is loaded using Hugging Face's `AutoTokenizer` and `AutoModelForSequenceClassification`. Input text is tokenized with Special Tokens (`[CLS]`, `[SEP]`) and passed through BERT's multi-head self-attention layers to compute token representations. PyTorch computes the output logits, which are then passed through a Softmax function to obtain probabilities for `POSITIVE` and `NEGATIVE` sentiment classes.

3 Software and Hardware Requirements

3.1 Software Requirements

- **Operating System**: Linux / macOS / Windows
- **Language/Runtime**: Python 3.10+
- **Core Libraries**: `fastapi`, `uvicorn`, `transformers`, `torch`, `pydantic`
- **Frontend Stack**: HTML5, CSS3, JavaScript (Fetch API)

3.2 Hardware Requirements

- **Processor**: Intel Core i5 / AMD Ryzen 5 or higher
- **RAM**: Minimum 8 GB (16 GB recommended)
- **Storage**: $\sim$ 2 GB free space for deep learning dependencies and model weights
- **GPU**: Optional (NVIDIA CUDA support enabled; CPU execution fully supported)

4 Mathematical Formulae

4.1 Self-Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where Q, K, and V are Query, Key, and Value matrices derived from token embeddings, and d_k is the dimensionality of key vectors.

4.2 Softmax Classification Output

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$$

Where z_c is the logit score corresponding to sentiment class $c \in \{\text{POSITIVE}, \text{NEGATIVE}\}$.

5 Libraries Used

- `transformers`: Provides pretrained `AutoTokenizer` and `AutoModelForSequenceClassification` for BERT.
- `torch`: PyTorch tensor computation and deep learning framework for forward pass calculation.
- `fastapi`: High-performance web framework for constructing RESTful API endpoints.
- `uvicorn`: ASGI web server to host and run FastAPI applications.

6 Complete Source Code

6.1 Backend Application (`app.py`)

```python
import torch
import torch.nn.functional as F
from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel
from transformers import AutoTokenizer,
    AutoModelForSequenceClassification

app = FastAPI(title="BERT Sentiment Analysis API")

# Explicit BERT base model fine-tuned on SST-2
MODEL_NAME = "textattack/bert-base-uncased-SST-2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)
model.eval()

class TextRequest(BaseModel):
    text: str
```

```
19
20 app.mount("/static", StaticFiles(directory="static"), name="static")
21
22 @app.get("/")
23 def read_root():
24     return FileResponse("static/index.html")
25
26 @app.post("/api/analyze")
27 def analyze_sentiment(request: TextRequest):
28     text = request.text.strip()
29     if not text:
30         raise HTTPException(status_code=400, detail="Text cannot be
    empty.")
31
32     # Tokenize input sequence using BERT Tokenizer
33     inputs = tokenizer(text, return_tensors="pt", truncation=True,
    max_length=512)
34
35     # Perform forward pass through BERT model using PyTorch
36     with torch.no_grad():
37         outputs = model(**inputs)
38         logits = outputs.logits
39         probabilities = F.softmax(logits, dim=-1)
40         confidence, prediction = torch.max(probabilities, dim=-1)
41
42     label_map = {0: "NEGATIVE", 1: "POSITIVE"}
43     predicted_label = label_map[prediction.item()]
44     confidence_score = round(float(confidence.item()), 4)
45
46     return {"label": predicted_label, "score": confidence_score}
```

6.2 Frontend Markup (static/index.html)

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
    =1.0">
6     <title>BERT Sentiment Analysis</title>
7     <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10     <div class="container">
11         <h1>Sentiment Analysis with BERT</h1>
12         <p class="subtitle">Pretrained Transformer Model (DistilBERT)</
    p>
13
14         <div class="card">
15             <textarea id="inputText" placeholder="Type your text here
    to analyze sentiment..."></textarea>
16             <button id="analyzeBtn" onclick="analyzeSentiment()">
    Analyze Sentiment</button>
17             <div id="result" class="result hidden"></div>
18         </div>
19
20         <footer>
```

```
            <p><strong>Student:</strong> zain pawle | <strong>Roll No:</strong> 25dco08 | <strong>Batch:</strong> 3 | <strong>Class:</strong> TECO1</p>
        </footer>
    </div>

    <script>
        async function analyzeSentiment() {
            const text = document.getElementById('inputText').value;
            const resultDiv = document.getElementById('result');
            const btn = document.getElementById('analyzeBtn');

            if (!text.trim()) {
                resultDiv.className = 'result error';
                resultDiv.innerText = 'Please enter some text!';
                resultDiv.classList.remove('hidden');
                return;
            }

            btn.disabled = true;
            btn.innerText = 'Analyzing...';
            resultDiv.classList.add('hidden');

            try {
                const response = await fetch('/api/analyze', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ text })
                });

                const data = await response.json();
                if (response.ok) {
                    const isPos = data.label === 'POSITIVE';
                    resultDiv.className = `result ${isPos ? 'positive' : 'negative'}`;
                    resultDiv.innerHTML = `<strong>Sentiment:</strong> ${data.label}<br><strong>Confidence:</strong> ${(data.score * 100).toFixed(2)}%`;
                } else {
                    resultDiv.className = 'result error';
                    resultDiv.innerText = data.detail || 'An error occurred.';
                }
            } catch (err) {
                resultDiv.className = 'result error';
                resultDiv.innerText = 'Failed to connect to backend server.';
            } finally {
                btn.disabled = false;
                btn.innerText = 'Analyze Sentiment';
                resultDiv.classList.remove('hidden');
            }
        }
    </script>
</body>
</html>
```

6.3 Frontend Styles (static/style.css)

```css
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
    background: #f0f2f5;
    color: #333;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    padding: 20px;
}

.container {
    width: 100%;
    max-width: 600px;
}

h1 {
    font-size: 1.8rem;
    text-align: center;
    color: #1a202c;
}

.subtitle {
    text-align: center;
    color: #718096;
    margin-bottom: 20px;
    font-size: 0.95rem;
}

.card {
    background: #ffffff;
    padding: 24px;
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

textarea {
    width: 100%;
    height: 120px;
    padding: 12px;
    border: 1px solid #cbd5e0;
    border-radius: 8px;
    resize: none;
    font-size: 1rem;
    outline: none;
    transition: border-color 0.2s;
}

textarea:focus {
    border-color: #3182ce;
```

```
}

button {
    width: 100%;
    margin-top: 16px;
    padding: 12px;
    background-color: #3182ce;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background-color 0.2s;
}

button:hover {
    background-color: #2b6cb0;
}

button:disabled {
    background-color: #a0aec0;
    cursor: not-allowed;
}

.result {
    margin-top: 20px;
    padding: 14px;
    border-radius: 8px;
    text-align: center;
    font-size: 1.05rem;
}

.result.positive {
    background-color: #c6f6d5;
    color: #22543d;
    border: 1px solid #9ae6b4;
}

.result.negative {
    background-color: #fed7d7;
    color: #742a2a;
    border: 1px solid #feb2b2;
}

.result.error {
    background-color: #feebc8;
    color: #744210;
    border: 1px solid #fbd38d;
}

.hidden {
    display: none;
}

footer {
    margin-top: 24px;
    text-align: center;
```

```
    font-size: 0.85rem;
    color: #4a5568;
}
```

6.4 Unit Test Suite (test_app.py)

```
import pytest
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert "BERT Sentiment Analysis" in response.text

def test_analyze_positive():
    response = client.post("/api/analyze", json={"text": "I love this
    product, it works amazingly well!"})
    assert response.status_code == 200
    data = response.json()
    assert data["label"] == "POSITIVE"
    assert "score" in data
    assert data["score"] > 0.5

def test_analyze_negative():
    response = client.post("/api/analyze", json={"text": "This service
    is terrible and disappointing."})
    assert response.status_code == 200
    data = response.json()
    assert data["label"] == "NEGATIVE"
    assert "score" in data
    assert data["score"] > 0.5

def test_analyze_empty():
    response = client.post("/api/analyze", json={"text": "   "})
    assert response.status_code == 400
    assert response.json()["detail"] == "Text cannot be empty."
```

7 Output Screenshots

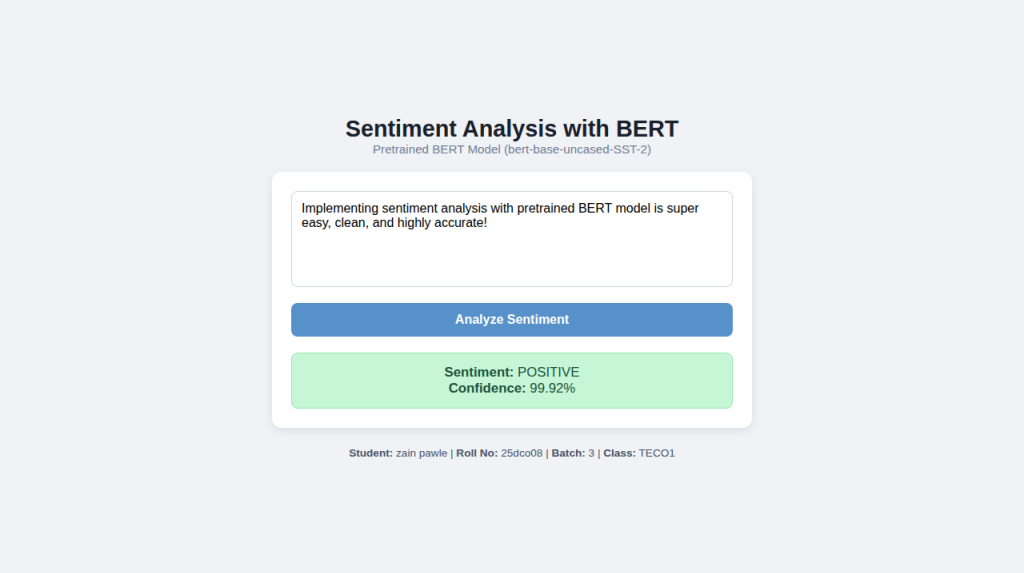

Figure 1: Positive Sentiment Output Example

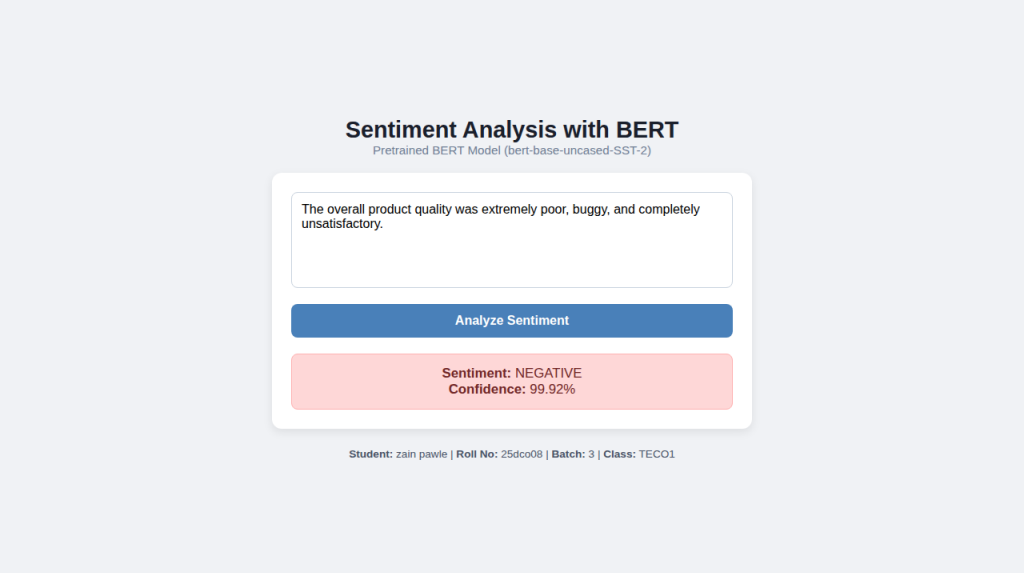

Figure 2: Negative Sentiment Output Example

8 Conclusion

The Sentiment Analysis web system was successfully implemented using a pretrained BERT model (`textattack/bert-base-uncased-SST-2`) paired with PyTorch, FastAPI, and a modern HTML/CSS interface.